

Server Set-up

This guide explains how to set up a Windows computer as a server. This should also work on Linux, but hasn't been tested on it.

Downloading SQLite:

Download the sqlite-tools for the appropriate OS from <https://www.sqlite.org/download.html>.

For Windows:

Precompiled Binaries for Windows

sqlite-dll-win-arm64-3510200.zip (1.03 MIB)	DLL for Windows ARM64, SQLite version 3.51.2. (SHA3-256: 28369c15e1ab475c1f54570a17474a9e1028bd25405893e78dcdc399b1ee2a78)
sqlite-dll-win-x64-3510200.zip (1.19 MIB)	DLL for Windows x64, SQLite version 3.51.2. (SHA3-256: d86617e984fc1d8163fb6e2bf363d9b674a35eecee2c0998dcd963817c5dae96)
sqlite-dll-win-x86-3510200.zip (1.03 MIB)	DLL for Windows x86 (32-bit), SQLite version 3.51.2. (SHA3-256: eb3d2bebc75707c283c7211aabb36826d46f5c1e71665ca430688dbc50e30938)
sqlite-tools-win-arm64-3510200.zip (5.29 MIB)	Command-line tools for Windows ARM64, including (1) the command-line shell , (2) sqldiff.exe , (3) sqlite3_analyzer.exe , and (4) sqlite3_rsync.exe . (SHA3-256: 1b8b8c83cfeddef0a9991028225d74b6c904dda05284f2237bd270c2465c1a11)
sqlite-tools-win-x64-3510200.zip (5.98 MIB)	Command-line tools for Windows x64, including (1) the command-line shell , (2) sqldiff.exe , (3) sqlite3_analyzer.exe , and (4) sqlite3_rsync.exe . (SHA3-256: d8f6cbab468c0b7a60fb7c1ebbeb1d14fda25326d0205472c218532266e85d0)

For Linux:

Precompiled Binaries for Linux

sqlite-tools-linux-x64-3510200.zip (3.99 MIB)	Command-line tools for Linux on x64, including (1) the command-line shell , (2) sqldiff , (3) sqlite3_analyzer , and (4) sqlite3_rsync . (SHA3-256: cd04740c391542745fec69f930714471e17d67dd454dc19fbd6d1be61ec822e)
--	---

You should have downloaded a .zip file. Go to your desired directory, and create a new folder called “SQLite” (I put mine inside my “c:\users\yourname” folder). Right-click the downloaded .zip and extract it into this folder. Check to make sure there are at least 3 files, “sqldiff.exe”, “sqlite3.exe”, and “sqlite3_analyzer.exe”.

Checking the installation:

Open up the Windows Command Prompt terminal (or the Linux equivalent), and navigate inside the “SQLite” folder you just created. If you are using Windows, type “.\sqlite3” into the terminal and press enter. Otherwise, if you are using Linux, type “sqlite3”. You should see the following:

```
PS C:\Users\weikai\SQLite> .\sqlite3
SQLite version 3.51.2 2026-01-09 17:27:48
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> |
```

At this point, SQLite has been installed successfully, and you can type “.quit” into the terminal to exit.

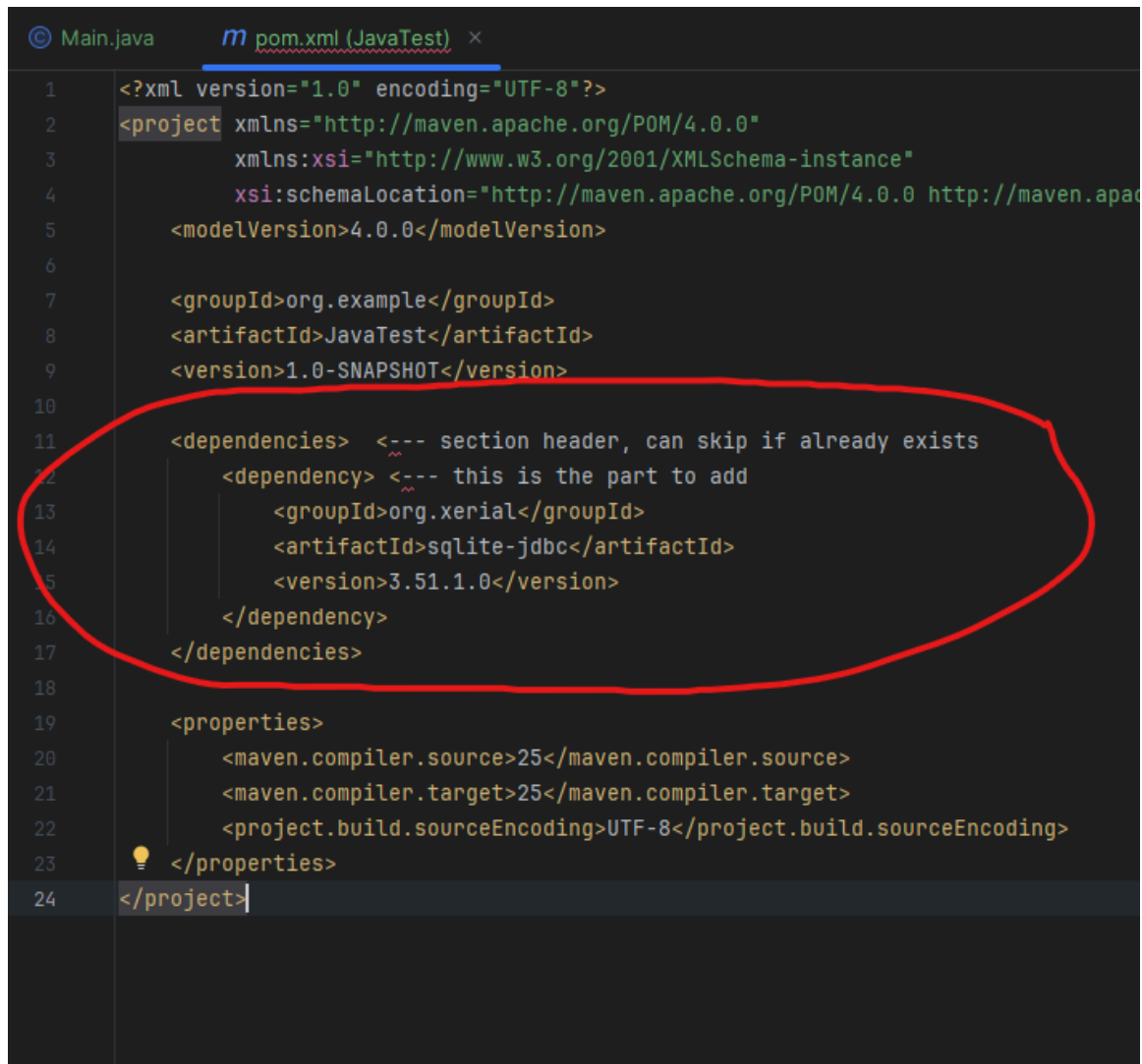
Setting up the JDBC driver (SKIP THIS):

SKIP THIS IF IMPORTING PROJECT FROM GIT. STILL NEED TO SET UP INTELLIJ MODULE.

To allow SQL to work with Java, a JDBC driver is required. Open pom.xml, and add a dependencies section above the properties section (if there isn't already one). In that section, add the following:

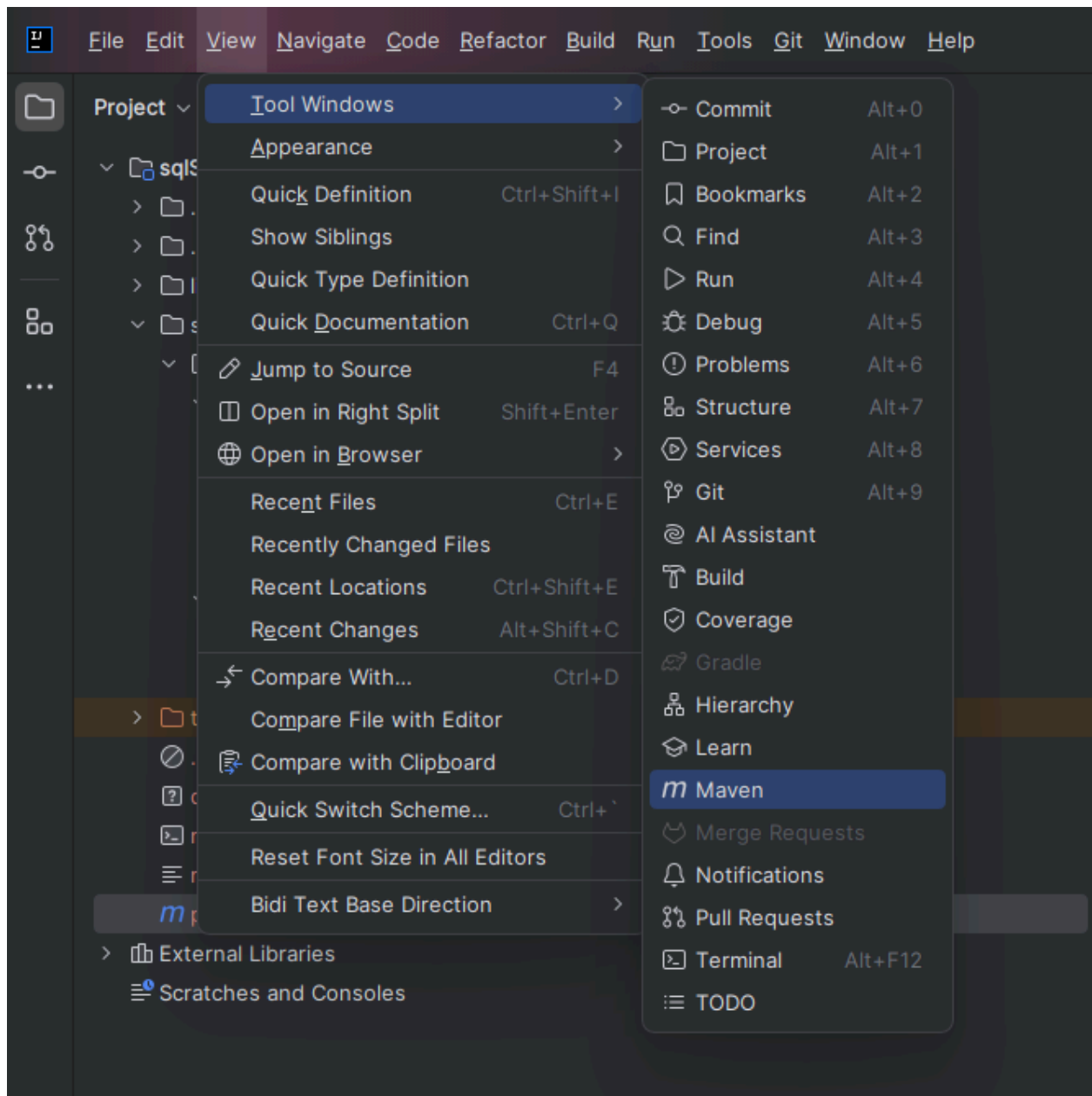
```
<dependencies>
  <dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.51.1.0</version>      //swap this out for the version you're using
  </dependency>
</dependencies>
```

Should look like this:

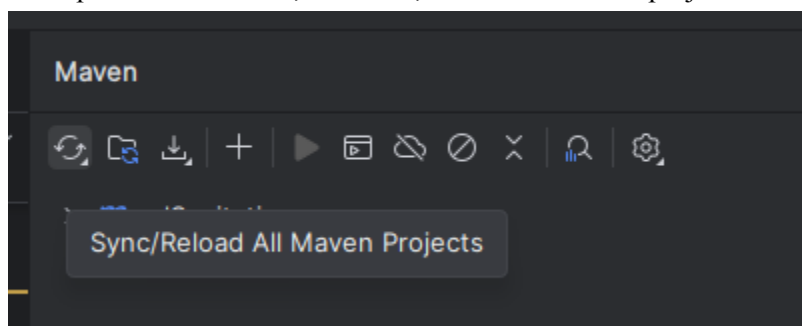


```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apac
5      <modelVersion>4.0.0</modelVersion>
6
7      <groupId>org.example</groupId>
8      <artifactId>JavaTest</artifactId>
9      <version>1.0-SNAPSHOT</version>
10
11     <dependencies> <!-- section header, can skip if already exists
12         <dependency> <!-- this is the part to add
13             <groupId>org.xerial</groupId>
14             <artifactId>sqlite-jdbc</artifactId>
15             <version>3.51.1.0</version>
16         </dependency>
17     </dependencies>
18
19     <properties>
20         <maven.compiler.source>25</maven.compiler.source>
21         <maven.compiler.target>25</maven.compiler.target>
22         <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
23     </properties>
24 </project>
```

Once done, must reload Maven



Will open a side window, click this, and choose reload project

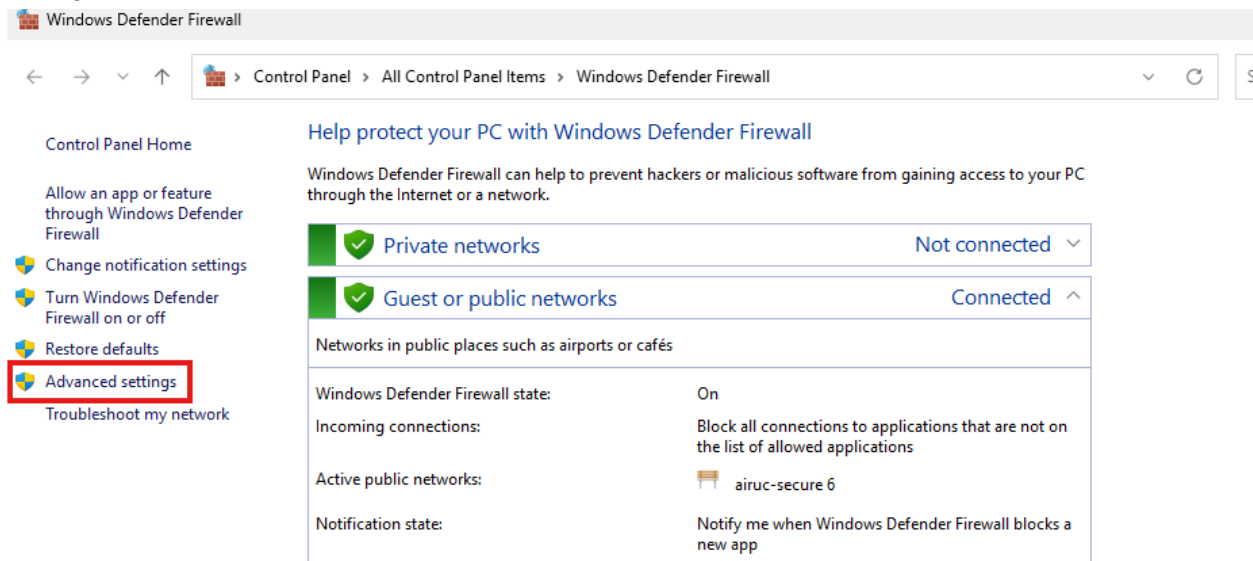


Running the server:

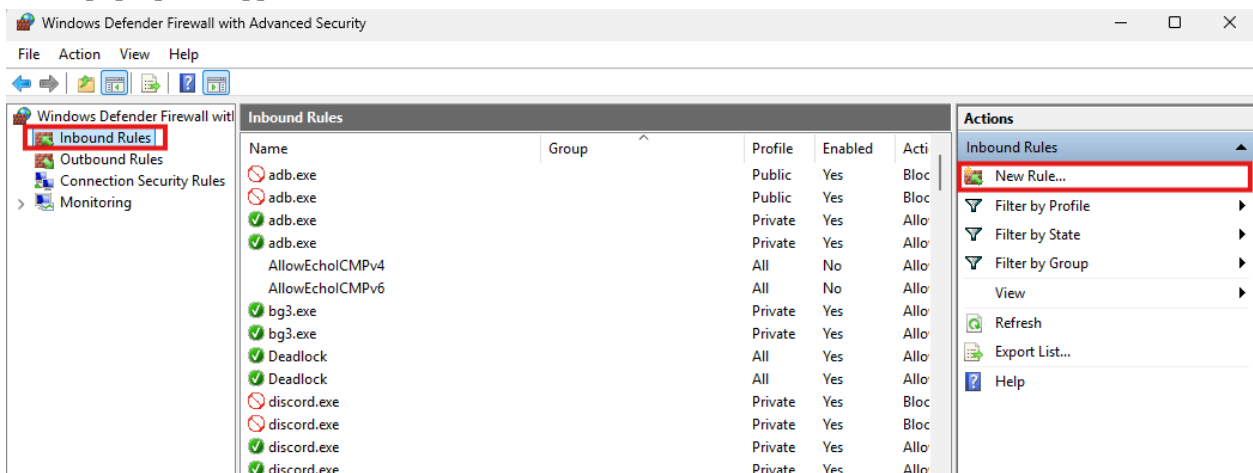
Unfortunately, the server cannot be run from the IntelliJ IDE like a normal program. Before the server can be run, the computer's firewall permissions have to be adjusted, and an artefact created.

Firewall Permissions:

The server listens on a specific port for connections. But for any connections to be made, a hole has to first be opened in the firewall to allow communication. To do this, **admin privileges are required**. Press the Windows key, and search for the Windows Defender Firewall app. Open it, and select the “Advanced Settings” tab.



A new pop-up will appear, select the “Inbound Rules” tab and click “New Rule...”.



Select the following options:

Rule type: Port

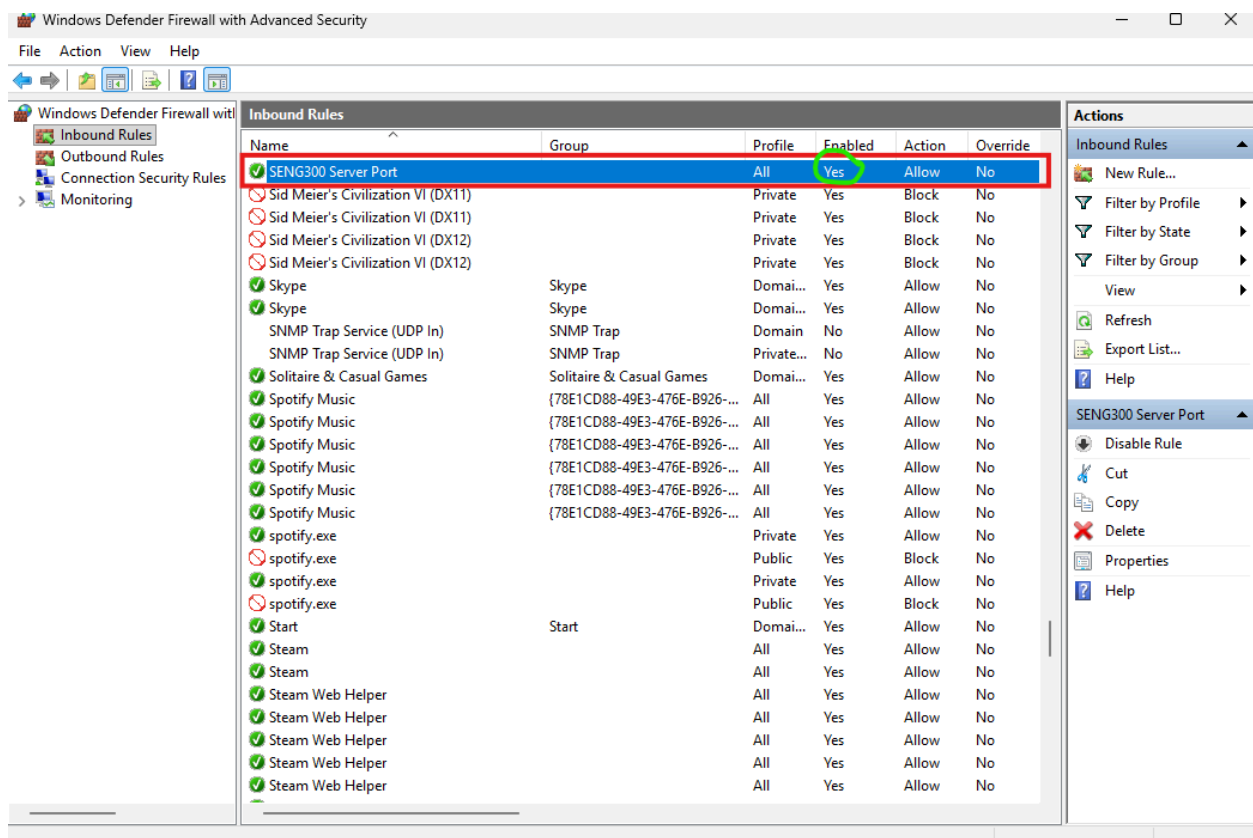
Protocol and Ports: TCP and specific local port. Put in the port number the server is being hosted on. You can check the port number being used inside the Database.java file.

Action: Allow the connection

Profile: Everything

Name: Up to you, choose a name and description that will allow you to find this rule later.

Once that is done, select finish. You should now see your rule in the list. Click on it, and make sure it is enabled.

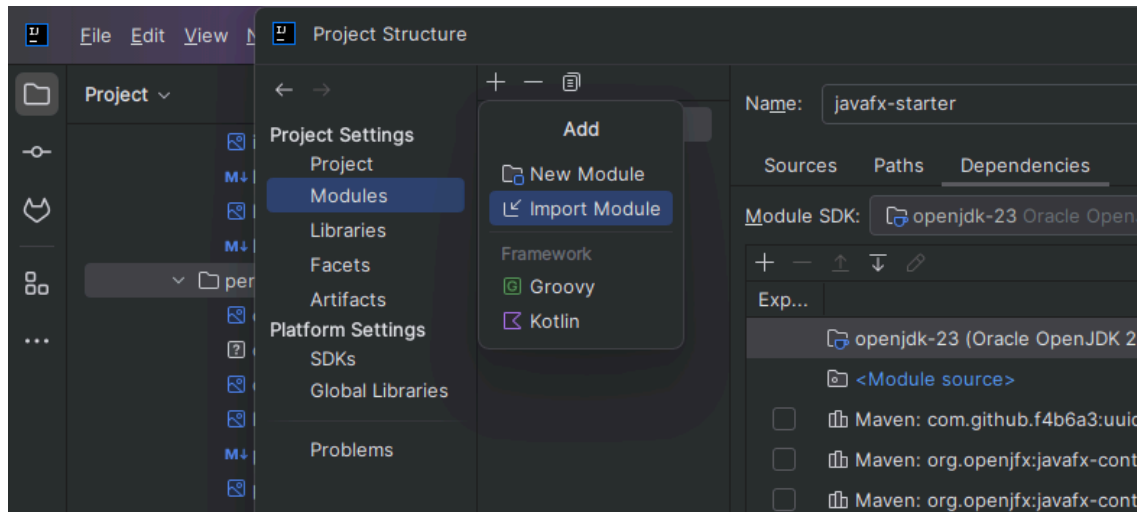


The newly added rule should be at the top of the list, but if you cannot find it you can sort the list by name. It should be enabled by default; if it is not, you can select the rule and enable it from the right sidebar.

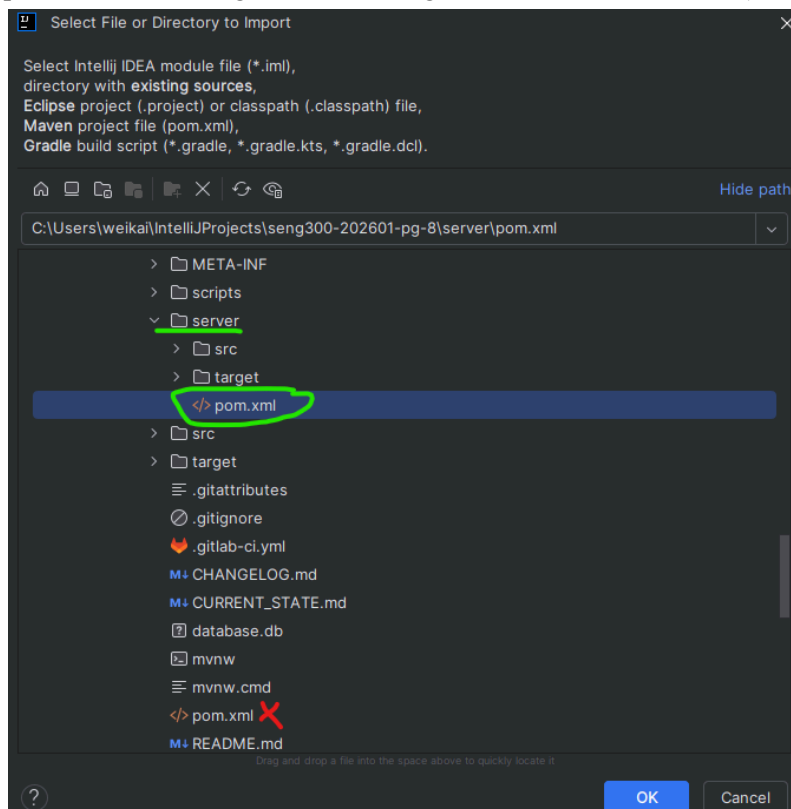
Then, select "Outbound Rules" from the left sidebar, and repeat the process. Once that is done, you should have 2 new firewall rules allowing access for the server's port number. One for inbound, and one for outbound. For security reasons, I recommend disabling these rules when the server is not running.

Setting up the IntelliJ module

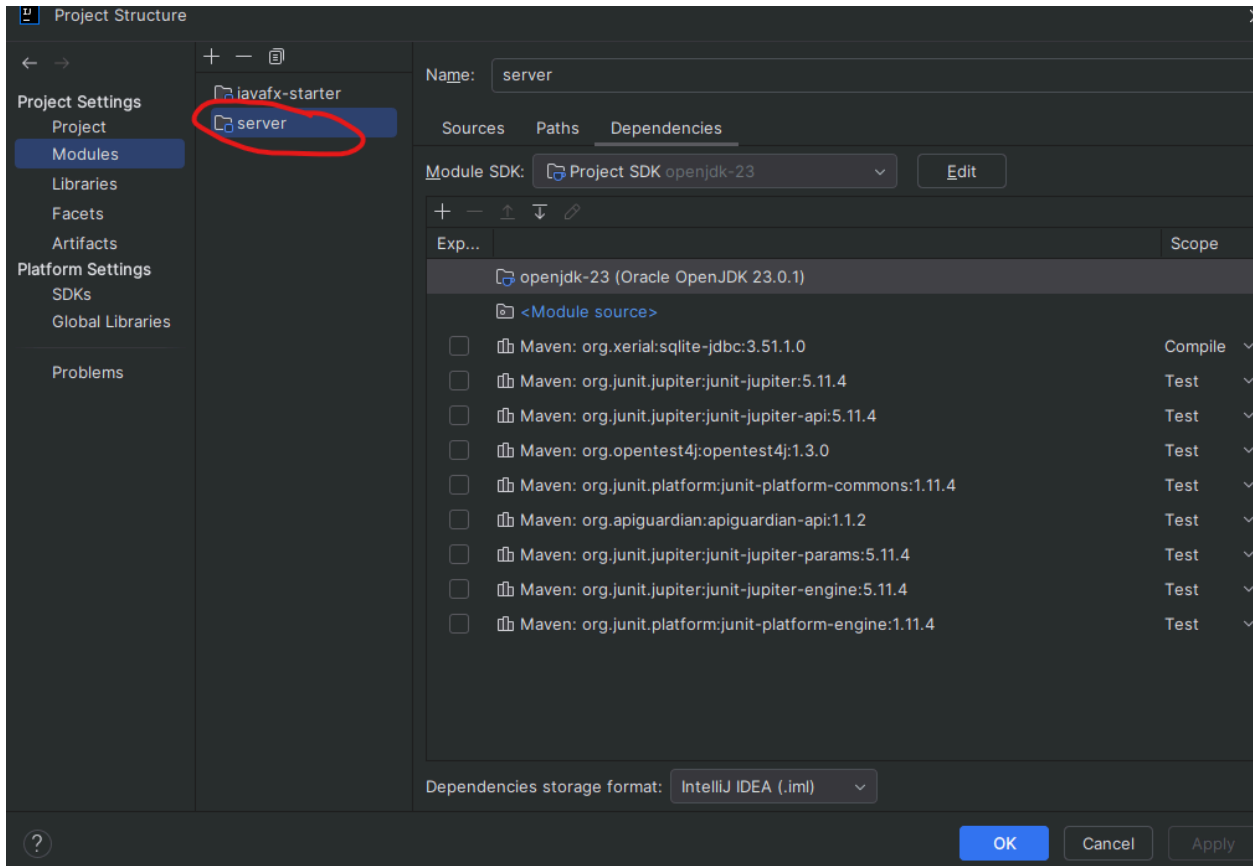
When pulling the project from git, IntelliJ modules are not automatically imported and have to be manually set up before running the server. Open the Project Structure tab, then select “Modules” > “+” > “Import Module”.



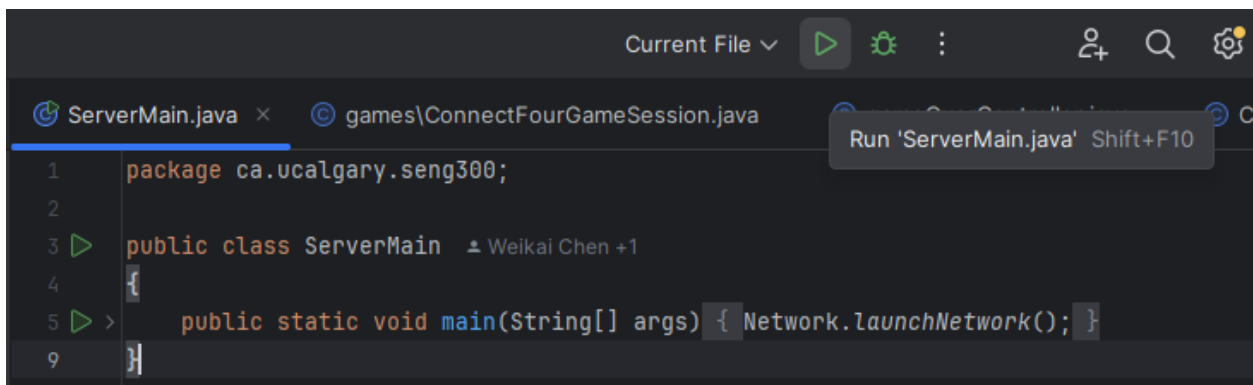
A new file browser tab will appear. Select the pom.xml INSIDE the server folder, not the one outside (the pom.xml circled in green in the image below, not the one in red)



Select OK. There should now be two entries in the module list.



Select Apply, then OK to close the Project Structure tab. IntelliJ should now recognise the ServerMain.java file as runnable. You can check this by navigating to the ServerMain.java file in IntelliJ and looking for the green play button in the top right. It should not be greyed out anymore.

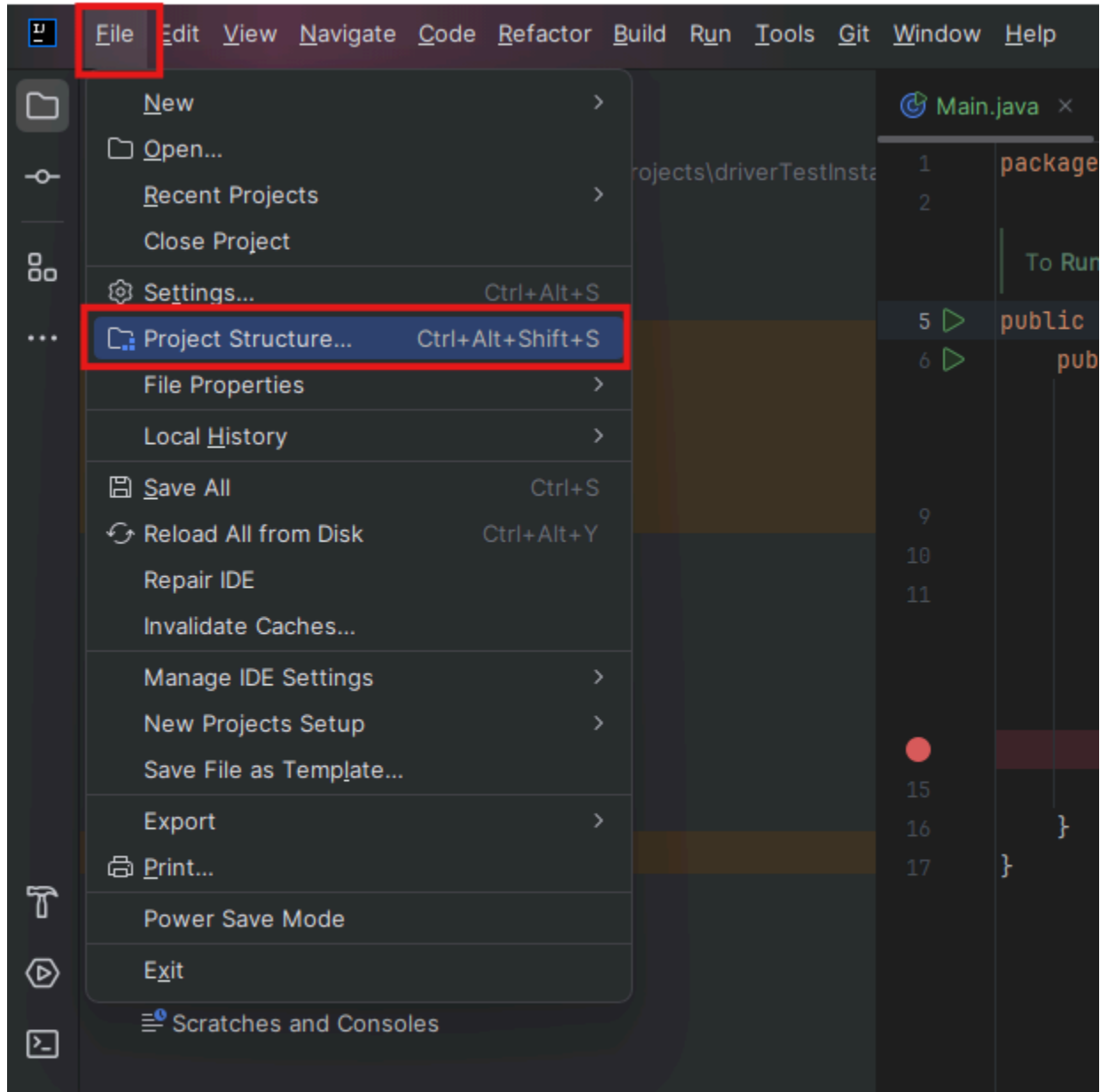


Creating an Artefact:

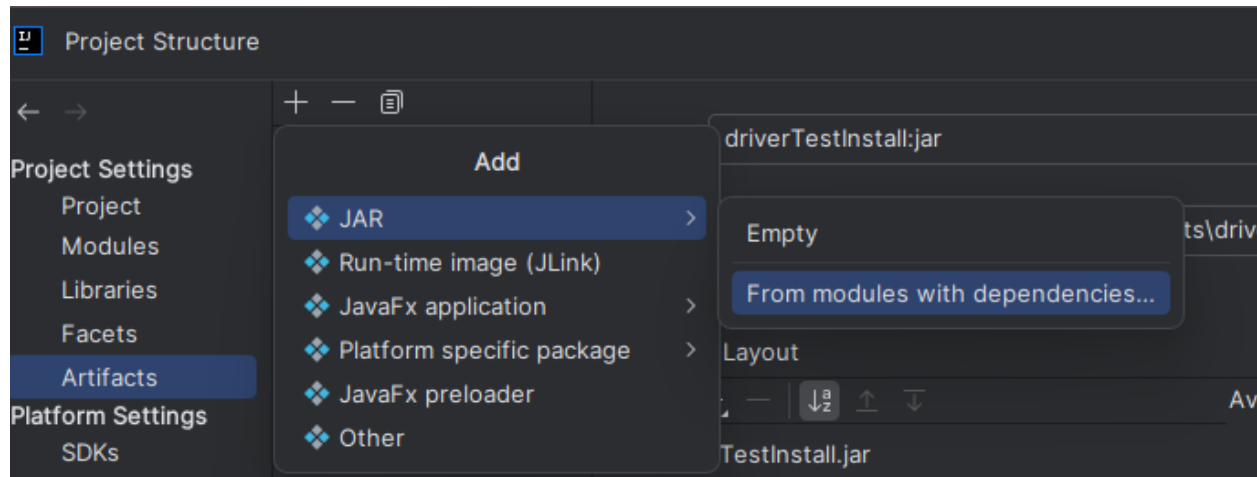
//not always needed, use netstat -an|find "14001" after running from IntelliJ to check if server is listening. If it is, can skip this next step and just run it from the IDE (using the green play button)

This program cannot run from inside the IDE. Even with firewall permissions for the port, the IntelliJ IDE doesn't have access. A workaround for this is to run the project as a .jar from the terminal instead. This next section will cover how to package the project as a .jar (i.e. artifact) and run it.

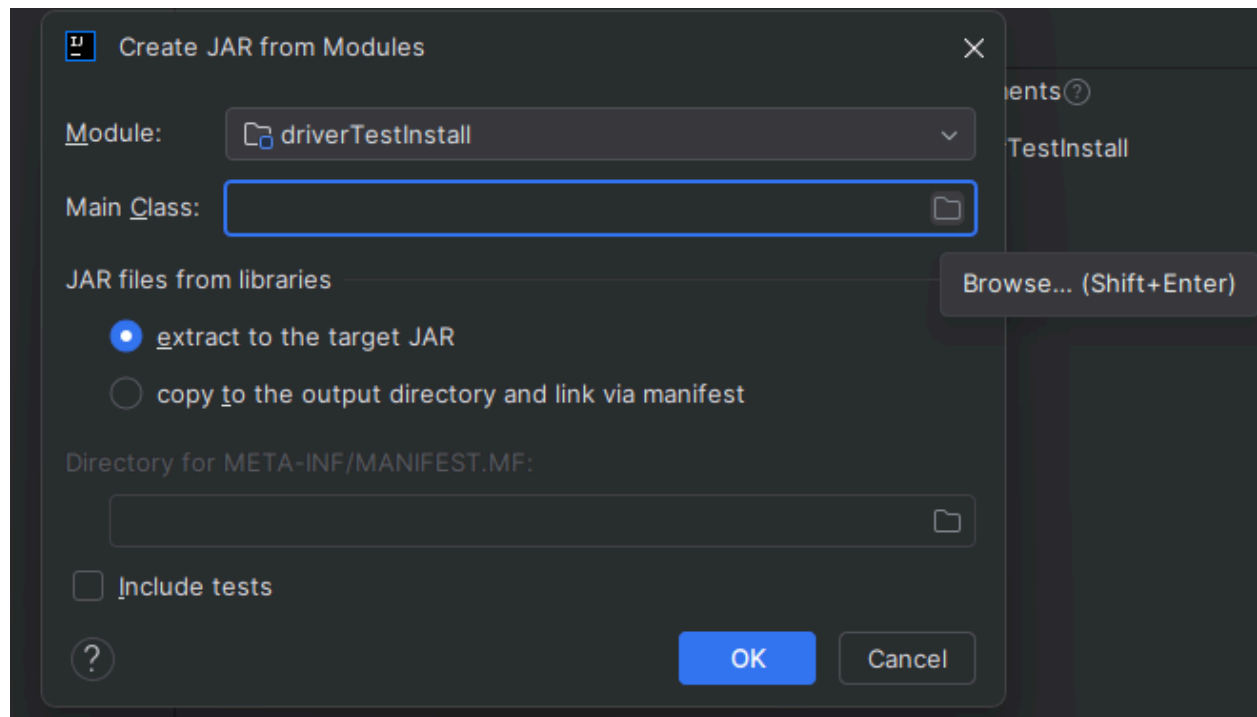
Open the server project files in IntelliJ. Select the "Project Structure" option from the top toolbar.



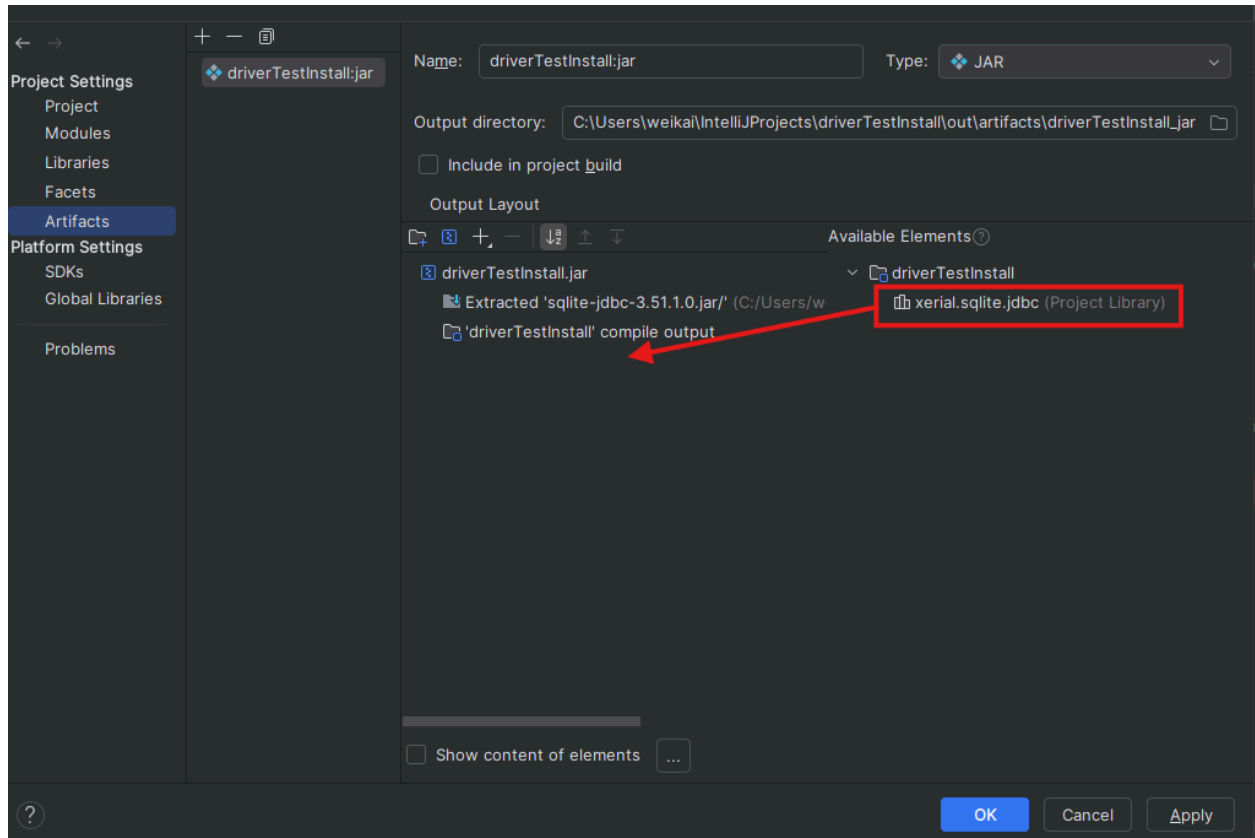
A new pop-up will appear. Select the “Artifacts” tab from the left, and then press the “+” sign. Choose the following options.



A new pop-up will appear. Select the browse option for the Main Class, and choose the method the server launches from from the list that appears.

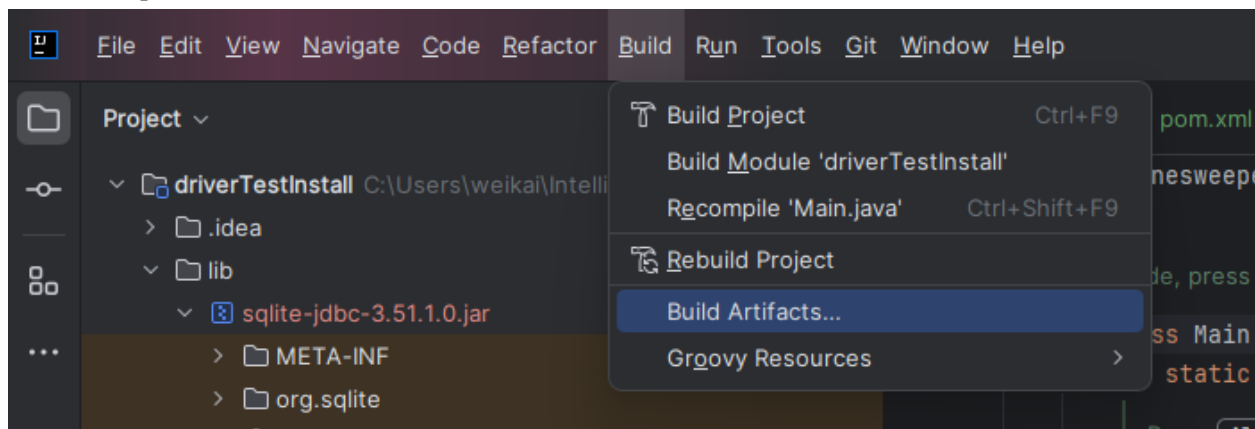


Select Ok. Check the newly created artifact; the JDBC driver might not be included by default. If it's not, double-click it to add. (It should move from the “Available Elements” section over to the left so it's under the .jar)

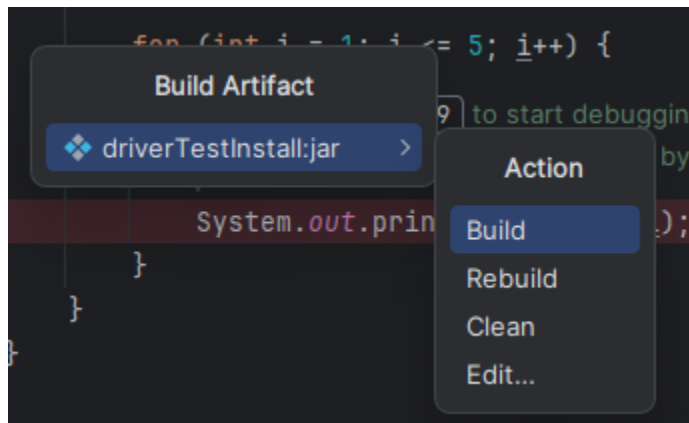


The next step is optional, but I highly recommend changing the file path of the .jar file. In the “Output Directory” field at the top, delete everything after the word “out”. This just removes some unnecessary subfolders for ease of access. Then click “Apply” and “Ok”.

From the top tab, select “Build” and “Build Artifacts”.



A new menu will appear. Select “Build”.



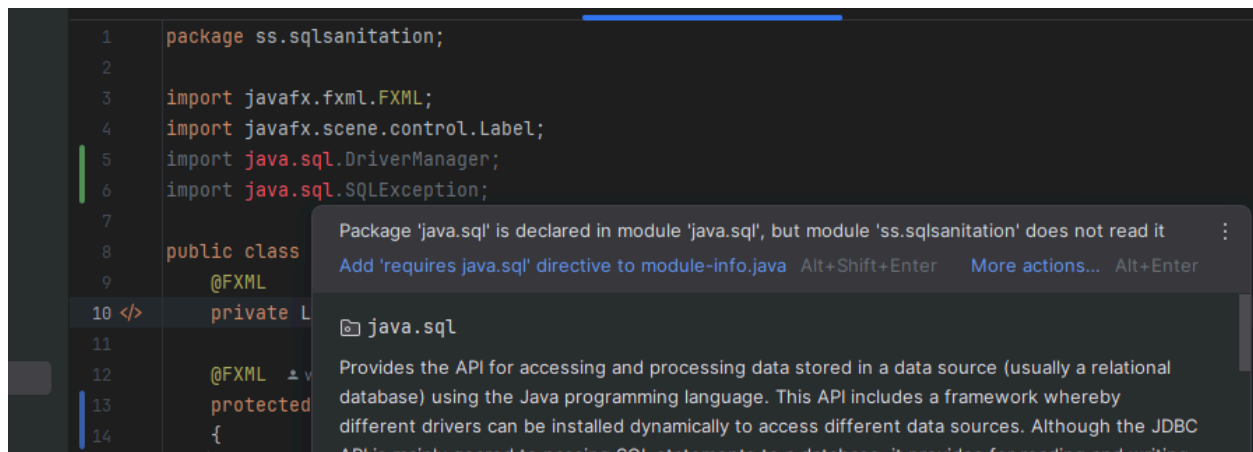
A new “out” subfolder will be created in the project files, with the .jar file inside.

To run the program, open the command terminal and navigate to the directory containing the .jar file. You can also use IntelliJ’s built-in terminal for this. Type “java -jar artifactName.jar” into the terminal to run the program.

The .jar file does NOT automatically update to reflect the changes made in the code. To update the .jar file, select Build > Build Artifacts > Rebuild. If at any point the .jar file is lost/deleted, a new one can be created anytime by choosing the “Build” option instead of “Rebuild”.

Trouble shooting:

When trying to import java.sql packages, may get an error like this:



Click the Add “requires java.sql” directive to module-info.java. Adds the line “requires java.sql;” to the module-info.java file.